

PRÁCTICA CONJUNTA

Estructuras de Datos + Metodología de la Programación

1. Introducción

En esta práctica se propone el desarrollo de un juego sencillo de turnos con interfaz gráfica en JavaFX, en el que un jugador se desplaza entre habitaciones conectadas formando un grafo.

El objetivo principal no es crear un videojuego complejo, sino diseñar e implementar un sistema software bien estructurado, aplicando:

- Diseño orientado a objetos
- Especificación formal/semi-formal
- Uso de estructuras de datos implementadas por el alumnado
- Persistencia de datos en JSON
- Desarrollo guiado por metodología

El trabajo se realizará en grupos de 3 personas.

El juego tiene un jugador que se mueve por una red de habitaciones. Cada habitación es una matriz de dimensiones arbitrarias y distintas entre ellas.

En cada posición de la matriz puede haber una trampa, un objeto, un enemigo, una puerta/escalera/portal/... que lleva a otra habitación (que se puede abrir o no... en función de si el jugador ha hecho o conseguido tal o cual cosa: esto es opcional), y otros elementos que el grupo decida.

El jugador ocupa una casilla del tablero de la habitación, y puede realizar acciones sobre las casillas adyacentes.

El objetivo es, a partir de una habitación inicial, conseguir salir de la red de habitaciones por la habitación/es que tengan puerta de salida al exterior.

Para ello el jugador dispondrá de un número de turnos. En un turno un jugador puede hacer un máximo de 1 movimiento y 1 acción.

En un turno todos los jugadores y enemigos de la habitación hacen sus movimientos y acciones por orden. Primero el jugador, luego el enemigo 1, luego el enemigo 2... hasta que todos los jugadores han completado sus posibilidades, y entonces se pasa de turno.

El movimiento se resolverá a partir del cálculo de la distancia máxima a la que se puede mover el jugador, mostrándole en cada turno hasta dónde puede llegar iluminando las celdas que son susceptibles de recibir al jugador. El jugador sólo puede elegir un destino de las casillas disponibles en cada turno (no puede hacer dos movimientos). En el cálculo de movimiento, se gasta 1 punto de movimiento por cada desplazamiento desde una posición a la posición inmediatamente superior, inferior, izquierda o derecha. El movimiento diagonal está prohibido de manera directa, y es necesario gastar un punto de movimiento en cada eje cartesiano.

El jugador tendrá unas características propias: puntos de vida, velocidad de movimiento (cuantas casillas puede llegar a moverse en un turno), fuerza o ataque (cuántos puntos de daño máximo hace a un enemigo), defensa (cuántos puntos es capaz de reducir el ataque de un enemigo), etc.

El jugador podrá coger objetos y usarlos (tendrá un inventario). Los objetos le otorgarán opcionalmente mejoras (poción con +10 puntos de vida, hacha con +5 puntos de ataque). Algunos objetos le otorgarán la mejora con solo portarlos, otros sólo cuando los equipe. Los objetos podrán opcionalmente tener un número de uso máximo, o un tiempo (en turnos o en tiempo real, a

elección del grupo) máximo.

Los enemigos tendrán también características similares a las del jugador. Su objetivo será moverse hacia el jugador para cortar el paso y atacarlo cuando lleguen a él o tengan un arma que les permita atacarlo a la distancia suficiente.

Los ataques se resolverán de la siguiente manera:

Vida disponible = Vida disponible – máximo(0,(ataque*(aleatorio*2)-defensa))

Donde vida disponible es la vida del defensor, máximo es una función que elige el valor más alto entre dos posibles, ataque es el valor de la característica del atacante (su ataque + modificadores por arma u otros elementos que os inventéis), aleatorio es un número aleatorio entre 0 y 1 que representa lo bien o mal que ataca, y defensa es el valor de la característica de defensa del defensor (defensa + modificadores por armadura u otros elementos que os inventéis).

En un turno se puede hacer como mucho un movimiento Y una acción:

- me muevo aquí y ataco
- me muevo aquí y uso
- no me muevo y uso
- me muevo aquí y no hago nada
- no me muevo y no hago nada

Siempre va primero el movimiento. Siempre mueve primero el jugador.

Después los enemigos de la habitación. Los enemigos no es necesario que pasen de una habitación a otra (elección del grupo).

Cuando el jugador abre una puerta, automáticamente se acaba el turno porque el jugador cambia de habitación al instante, y se vuelve a reiniciar el proceso.

Si el jugador llega a una de las puertas de salida del juego y la abre, ha ganado.

Si el jugador se queda sin turnos sin haber llegado a una puerta de salida, ha perdido.

Si el jugador pierde sus puntos de vida, pierde.

Opcional: además del contador de turnos general de la partida, cada habitación podría tener un contador de turnos propio, de forma que si no se consigue pasar la habitación dentro de esos turnos, hace que el jugador pierda la partida aunque le queden turnos de partida.

2. Objetivos de aprendizaje

2.1 Metodología de la programación

- Elaborar especificaciones de software
- Diseñar mediante UML
- Definir antes de implementar
- Validar mediante pruebas
- Uso responsable de herramientas de IA

2.2 Estructuras de datos

- Implementar estructuras de datos propias
- Seleccionar estructuras adecuadas para cada problema
- Aplicarlas en un sistema real
- Modelar grafos y estructuras internas

3. Descripción del sistema a desarrollar

El sistema consiste en un juego de exploración por habitaciones.

3.1 Conceptos principales

Jugador

Tiene:

- Vida
- Ataque
- Defensa
- Capacidad de movimiento
- Inventario de objetos
- Posición dentro de la habitación

Habitaciones (IMPORTANTE)

Cada habitación NO es solo un nodo abstracto, sino que contiene una estructura interna en forma de matriz.

Especificación:

- Cada habitación es una matriz bidimensional de tamaño definido por los alumnos
- Cada celda de la matriz puede contener:
 - Un enemigo
 - Un objeto
 - Un elemento interactivo (puerta, trampa, etc.)
 - Estar vacía
- El jugador ocupa una posición concreta dentro de la matriz

Consecuencias:

- El movimiento del jugador tiene dos niveles:
 1. Movimiento dentro de la habitación (matriz)
 2. Movimiento entre habitaciones (grafo)
- Para cambiar de habitación, el jugador deberá estar en una celda que represente una salida

Representación en interfaz

Cuando el jugador se encuentra en una habitación:

- Se debe representar la matriz completa
- Se deben visualizar:
 - Posición del jugador
 - Objetos
 - Enemigos
 - Elementos interactivos

Sugerencia:

- Usar una rejilla (GridPane) en JavaFX
-

Objetos al inventario.

Ejemplos:

- Armas
- Pociones
- Llaves
- Objetos especiales

Ciertos objetos serán “equipables” para realizar ciertas acciones automáticamente: Si equipo un arma (la elijo como activa) me permitirá atacar y usar el arma en ese ataque. Si equipo un escudo (lo elijo como activo) se me permitirá defenderme con él. Los elementos sólo se pueden equipar hasta cierto punto, y se equipan en ciertas zonas: sólo tengo dos brazos, sólo tengo 10 dedos, sólo tengo un torso, sólo tengo dos manos... los elementos irán a equiparse a los puntos correspondientes.

Si quiero abrir una puerta, y la puerta no necesita llave, podré abrirla inmediatamente, pero si necesita llave, necesitaré equipar primero la llave para poder usarla al abrir la puerta.

Ciertos objetos serán fungibles: si me bebo una pócima que me da +20 puntos de vida, esa pócima desaparecerá.

Enemigos

- Movimiento
- Vida
- Ataque
- Defensa
- Posición dentro de la habitación

Un enemigo siempre tiene equipado aquello con lo que ataca y se defiende.

Acciones del jugador

- Moverse dentro de la habitación (sólo 1 movimiento por turno)
- Cambiar de habitación
- Recoger objetos (sólo 1 por turno)
- Usar objetos (sólo 1 por turno)
- Atacar enemigos (sólo 1 por turno)
- Defenderse es una acción automática, y siempre se realiza ante un ataque: ejemplo: si tengo dos monstruos que se mueven hacia mí, y me atacan, mi personaje SIEMPRE se defiende sin que yo tenga que hacer nada.

4. Requisitos funcionales mínimos

El sistema debe permitir:

1. Crear un grafo de habitaciones
2. Definir cada habitación como una matriz
3. Calcular el camino mínimo desde la posición del jugador hacia la puerta de la habitación adecuada para conseguir el camino mínimo desde la habitación de inicio del juego hasta la salida de la habitación final del juego: tanto la distancia a la puerta adecuada como el número mínimo de

habitaciones para salir se le informarán al jugador constantemente, pudiendo “comprar” el jugador ver el camino en pantalla.

4. Crear objetos y sus modificadores.
5. Posicionar objetos en las habitaciones.
6. Mover al jugador dentro de la habitación
7. Mover al jugador entre habitaciones
8. Mostrar estado del jugador
9. Gestionar inventario
10. Interactuar con objetos
11. Combatir enemigos
12. Definir condición de victoria
13. Definir condición de derrota
14. Guardar partida en JSON
15. Cargar partida desde JSON
16. Cargar configuración inicial desde JSON

5. Interfaz gráfica (JavaFX)

Debe incluir:

- Representación de la habitación como matriz
- Panel de información del jugador
- Lista de acciones disponibles
- Registro de eventos del juego
- Elementos interactivos básicos

Importante:

- No se requiere uso de gráficos complejos
- Se valorará claridad sobre estética

6. Persistencia en JSON

Se deben manejar dos tipos de datos:

Configuración de partida

Incluye:

- Grafo de habitaciones
- Dimensiones de cada matriz
- Contenido de cada celda
- Posición inicial
- Objetivo del juego

Estado de partida

Incluye:

- Posición del jugador (habitación + coordenadas)
- Inventario
- Estado de enemigos
- Objetos recogidos
- Estado del juego

7. Restricciones de estructuras de datos

NO se pueden utilizar:

- ArrayList
- HashMap
- LinkedList
- ni estructuras estándar equivalentes

Sí se deben implementar:

- Listas enlazadas
- Pilas
- Colas
- Listas circulares
- Árboles
- Grafos
- Cualquier otra estructura de datos que se necesite.

7.1 Requisito clave

- El mapa debe ser un grafo propio
- La matriz de habitaciones debe implementarse con estructuras propias

7.2 Justificación obligatoria

Se debe documentar:

- Qué estructura se usa
- Por qué
- Coste de operaciones

8. Metodología y especificación

Antes de programar se debe realizar:

8.1 Requisitos

- Funcionales
- No funcionales

8.2 Casos de uso

Ejemplos:

- Mover dentro de habitación
- Cambiar de habitación
- Atacar enemigo

Especificaciones para los casos:

Mover dentro de habitación

- Precondición: celda destino válida
- Postcondición: nueva posición actualizada

Cambiar de habitación

- Precondición: celda es salida
- Postcondición: jugador cambia de nodo del grafo

8.3 Modelo de dominio

Clases sugeridas:

- Jugador
- Habitación
- Celda
- Objeto
- Enemigo
- Grafo

8.4 Contratos

Declarar las interfaces necesarias, tanto de juego como de estructuras de datos.

8.5 Invariantes

- El jugador siempre ocupa una celda válida
- Una celda no contiene múltiples entidades. (si atacas, lo haces a la celda contigua; si coges un objeto, lo coges de la celda contigua, si pasas por una celda con una trampa, caes en la trampa, y la trampa desaparece, para abrir una puerta, lo haces desde la contigua, para atravesar una puerta, lo haces poniéndote en la celda de la puerta).
- Un objeto no puede estar duplicado, puede haber dos iguales.
- La vida nunca es negativa

9. UML obligatorio

Se deben incluir:

- Diagrama de casos de uso
- Diagrama de clases
- Diagrama de secuencia (mínimo uno)
- Diagrama de estados del juego (mínimo uno)
- Diagrama de actividad (lógica de la operación del jugador, al menos uno)

10. Diseño previo de interfaz (MUY IMPORTANTE)

Como ayuda para centrar el desarrollo:

Se requiere que los alumnos diseñen previamente las pantallas del sistema antes de programar.

Este diseño debe incluir:

- Bocetos (papel o digital)
- Distribución de elementos:
 - Zona de mapa (matriz)
 - Zona de estado del jugador
 - Zona de acciones
 - Zona de mensajes

Objetivo

- Organizar ideas
- Definir flujo de interacción
- Evitar empezar directamente en JavaFX sin planificación

Importante:

No se trata de implementar todavía, sino de pensar y diseñar.

Sugerencia: este debería ser el primer paso, para que discutan todos los miembros del grupo como “ven” el juego y así poder tener una visión única y cohesionada para el desarrollo. Recordad: sois un equipo, todos debéis ir alineados para conseguir el éxito de la práctica.

11. Uso de Inteligencia Artificial

Permitido con condiciones:

- Solo tras especificación
 - Registro obligatorio de uso
 - Incluir:
 - Agentes
 - Skills
 - Prompts
 - Resultados
 - Modificaciones
 - Análisis y evaluación de lo conseguido: Crítica al uso hecho y acciones de mejora
 - Todo debe poder explicarse
-

12. Pruebas

Se deben incluir:

- Pruebas de estructuras de datos
 - Pruebas de lógica del juego
 - Pruebas de carga y guardado de JSON
 - Logs del sistema
 - Gestión de excepciones
-

13. Entregables

1. Código fuente (repositorio GitHub + ZIP)
2. Documento de diseño
3. UML
4. JSON de ejemplo
5. Pruebas
6. Diario de IA
7. Bocetos de interfaz

Recordad: una entrega en cada asignatura!!! (es obligatorio!)

15. Ampliaciones (opcional)

- Podéis ampliar el juego tanto como queráis.
-

16. Consejos

- Diseñar antes de programar
- Empezar por estructuras de datos, reutilizando las clases de prácticas previas, y estudiando cómo usarlas para las estructuras de datos de este

juego.

- Separar lógica de interfaz
- Probar por partes
- No empezar por JavaFX

17. Autoevaluación

Antes de entregar:

- ¿Hemos implementado nuestras estructuras?
- ¿El grafo funciona correctamente?
- ¿La matriz de habitaciones está bien modelada?
- ¿El jugador se mueve correctamente dentro y entre habitaciones?
- ¿Se representa correctamente en interfaz?
- ¿Se guarda y carga bien?
- ¿Tenemos UML coherente?
- ¿Las pruebas son suficientes?
- ¿Hemos documentado la IA?
- ¿Todos entienden el sistema?

Importante:

Se valorará más un sistema claro, correcto y bien diseñado que uno complejo pero desorganizado.

1. **Manejo de Errores:** El programa debe gestionar adecuadamente situaciones inesperadas (ej. intentar mover fuera del tablero, atacar a un aliado, errores al leer/escribir el archivo JSON) mediante excepciones.
2. **Log:** Todas las operaciones del juego deberán recogerse en un log, donde aparecerán todas las operaciones realizadas. Ejemplo: El usuario recoge un bastón que suma +2 posiciones de movimiento.
3. **Visualización:** Al final de la partida debe mostrarse el log generado. Debe poder verse el inventario constantemente. El usuario, en su fase de acción del turno, tiene que poder usar elementos del inventario.

Evaluación:

Parte 1: Metodología de la Programación

1. Diseño Orientado a Objetos:

- Correcta identificación y modelado de clases.
- Uso adecuado de encapsulamiento (atributos privados, métodos públicos/protegidos).
- Aplicación de herencia y polimorfismo (clases base/derivadas para unidades, acciones).
- Cohesión y bajo acoplamiento entre clases.
- Claridad y organización general del código fuente.
- Diseño UML, diseño visual.

2. Gestión de Excepciones:

- Identificación de posibles puntos de error (movimientos inválidos, ataques inválidos, E/S de archivos, etc.).
- Uso correcto de bloques try-catch-finally.
- Creación de excepciones personalizadas si es relevante (ej. `InvalidMoveException`).
- Robustez general de la aplicación ante errores.

3. Entrada/Salida con JSON:

- Correcta serialización del estado del juego a formato JSON.
- Correcta deserialización del estado del juego desde JSON para cargar la partida.
- Manejo de posibles errores durante la lectura/escritura de archivos.
- Integridad de los datos guardados y cargados.
- Gestión de habitaciones y grafos

4. Interfaz de Usuario con JavaFX:

- Funcionalidad básica de la interfaz (visualización correcta, interacción con el usuario).
- Separación adecuada entre la lógica del juego y la interfaz (Modelo-Vista-Controlador básico o similar).
- Respuesta a eventos del usuario (clics en elementos / jugador, botones).
- Claridad y usabilidad de la interfaz (aunque sea sencilla): Correcta segmentación del juego en pantallas y secciones (paneles), uso de grids, botones...

5. Metodología de desarrollo con agentes de IA:

- Correcto uso de los agentes de IA en un flujo de trabajo de desarrollo de software.
- Correcta especificación de requisitos, metodología de trabajo y gestión del ciclo de vida del proyecto.
- Diario de utilización del agente de IA: configuración, operaciones, resultados, crítica, reajustes y metodología final extraída por el grupo para un uso satisfactorio.

Parte 2: Estructuras de Datos

(Importante: Se evaluará el uso de las implementaciones PROPIAS de las estructuras (no usar `java.util.*` para las estructuras evaluadas) de Listas, Pilas, Colas, Árboles y Grafos. Usar estructuras de java o de librerías de terceros conlleva una calificación de 0 puntos en la asignatura de Estructuras de Datos)

1. Implementación de Estructuras Fundamentales (Listas, Pilas, Colas):

- Correcta implementación de las operaciones básicas de las listas (añadir, eliminar, buscar) usadas para gestionar los elementos del juego, o el inventario.
- Correcta implementación y uso de una cola para gestionar el orden de los turnos.
- (Opcional) Uso de una pila si implementa alguna funcionalidad de "deshacer" acción.
- Eficiencia básica de las implementaciones.

2. Implementación y Uso de Grafos:

- Correcta implementación de la estructura de grafo (dirigido o no dirigido, ponderado si el terreno tiene costes de movimiento, anotado si tiene más elementos...).
- Aplicación de algoritmos sobre el grafo:
 - **Búsqueda en Anchura (BFS):** Para determinar las casillas alcanzables dentro del rango de movimiento de una unidad.
 - **Algoritmo de Dijkstra:** Si se implementan costes de movimiento variables por habitación, para encontrar la ruta óptima.
 - **Búsqueda:** Para encontrar unidades enemigas cercanas o planificar movimientos básicos.

3. Implementación y Uso de Árboles:

- Si se añaden mecánicas más complejas como árboles de habilidades, evaluar la implementación (árbol binario, árbol de búsqueda) y su uso.
- Si se usa un árbol para organizar tipos de unidades o elementos de forma jerárquica (ampliación).
- Uso de árboles para elegir las posibles acciones de la partida.

4. Integración y Elección de Estructuras:

- Justificación de por qué se eligió cada estructura de datos para resolver un problema específico dentro del juego.
- Correcta integración de las estructuras de datos con la lógica del juego.
- Eficiencia general de las soluciones basadas en las estructuras de datos elegidas (evitar usos claramente ineficientes).

Condiciones generales para las entregas:

- Se espera que los alumnos añadan test unitarios con JUnit para todas las clases no visuales.
- Debe generarse una memoria en formato PDF explicando el proyecto. La memoria debe, obligatoriamente, con una sección denominada “crítica del proyecto”. En esta sección los alumnos indicarán las mejoras y fallos del proyecto realizado.
 - El enunciado deja abiertos algunos puntos. Los alumnos **pueden y deben** tomar las decisiones que consideren para completar el proyecto propuesto. Estas decisiones mostrarán la capacidad, visión y planificación de los alumnos en tareas complejas. Se espera que estas decisiones se traten y justifiquen en la memoria.
- Se pueden reutilizar desarrollos realizados previamente en la asignatura.
- Debe generarse un vídeo explicando el proyecto y mostrando TODAS las funcionalidades desarrolladas, donde aparezcan TODOS los alumnos que han desarrollado el proyecto (usando una webcam, mediante videoconferencia o similar). **IMPRESINDIBLE.**
 - El vídeo deberá subirse a la plataforma Blackboard como un fichero de menos de 100 Mb, o múltiples ficheros de menos de 100 Mb (usando una utilidad tipo WinRAR o similar). Adicionalmente, podrá subirse el vídeo a Youtube y adjuntar el enlace (importante: adicionalmente, si el vídeo no se sube también a la plataforma Blackboard, no se tendrá en cuenta).
- Uso de herramientas de IA:
 - Esta práctica está pensada para que el grupo la desarrolle **utilizando agentes de IA** según la metodología explicada en clase. Eso no quita para que los alumnos deban revisar el código generado, y se aseguren de que es correcto, no sólo funcionalmente, sino en su estructura y operación.
- **En la portada de la memoria constarán los nombres de los alumnos que han realizado la práctica.** Si el nombre de algún alumno no aparece en la portada de la memoria, se entenderá que no ha colaborado en su realización de manera suficiente, y ese alumno tendrá una nota de cero puntos.
- **Entrega límite:** vía Blackboard el 28/5/2025 a las 10h, una entrega para cada asignatura.